The engine of the donation game with scores

What each piece does, how, with which parameters, and an example

October 18, 2026

Contents

1 The model and how the engine is organised **1**
- 1.1 The model in one page . . . 1
- 1.2 Notation and the names in the code . . . 2
- 1.3 How the pieces fit together . . . 2

2 Elements: classes, strategies, update rules **3**
- 2.1 Score partitions . . . 3
- 2.2 Strategies . . . 3
- 2.3 Scoring norms (the update rules) . . . 6
- 2.4 Noise . . . 8

3 Score dynamics: from strategies and a norm to stable score distributions **9**
- 3.1 Population . . . 9
- 3.2 Transition rates . . . 10
- 3.3 The birth–death chain . . . 11
- 3.4 Stable score distributions . . . 12

4 Dynamics: payoffs, replicator dynamics, rest points and ESS **13**
- 4.1 Payoffs . . . 13
- 4.2 Replicator dynamics . . . 14
- 4.3 The strategy simplex and its rest points . . . 15
- 4.4 Evolutionarily stable strategies . . . 17

A Where the pieces are used, and how to rebuild this document **19**

1 The model and how the engine is organised

1.1 The model in one page

The game. Individuals meet in pairs. In each meeting one is the *Can* (the donor) and the other the *Need* (the receiver): each individual can help with probability q, a given ordered assignment of the two roles therefore has probability $\kappa = q(1-q)$ (the *role factor*, 1/4 for $q = \frac{1}{2}$). The donor chooses whether to help. Helping costs the donor c and gives the receiver b.

Scores. Every individual has an integer score $j \in \{0, 1, 2, \dots\}$ (a reputation; there is no cap). After a meeting the *scoring norm* changes the donor's score, and the receiver's, by -1, 0 or $+1$, depending on whether help was given and on the *classes* of the two scores. A class is a set of scores: for binary norms $s = 0$ and $s \geq 1$, for ternary norms $s = 0$, $0 < s \leq \tau$ and $s > \tau$. A score of 0 cannot go down.

Strategies. A strategy says with which probability a donor helps. It may depend on the receiver's class and on the donor's own score. A population holds n strategies with frequencies ρ_i (summing to 1). Each strategy i has its own score distribution $w_j^{(i)}$ = the fraction of its individuals with score j.

Score chains. Seen from one individual of strategy i, the score is a birth–death chain on $\{0, 1, \dots\}$: it moves up from $j-1$ to j at rate $g_j^{(i)}$ and down from j to $j-1$ at rate $f_j^{(i)}$. The rates come from the norm, the strategies, who is in the population and, through noise ε, from random score changes. They depend on the score distributions of *all* strategies, so the n chains are run together until the distributions stop changing (the *stable* score distributions).

Payoffs, replicator dynamics, ESS. With the stable distributions, the probability that a Can of strategy i helps a Need of strategy k is H_{ik}, and the expected payoff of strategy i is

$$\pi_i \;=\; \kappa\Big(b\sum_k \rho_k H_{ki} \;-\; c\sum_k \rho_k H_{ik}\Big).$$

The frequencies then evolve by the replicator equation $\dot{\rho}_i = \rho_i(\pi_i - \bar{\pi})$, and a strategy is an *evolutionarily stable strategy* (ESS) if a rare invader cannot do better than it.

1.2 Notation and the names in the code

symbol	in the code	meaning
j	`j`, a column of `w`	a score, $0, 1, 2, \dots$
i, k	a row of `w`, `rho`	a strategy
ρ_i	`rho[i]`	frequency of strategy i
$w_j^{(i)}$	`w[i, j]`	fraction of strategy i with score j (array of shape $n \times L$)
κ	`role_factor`	$q(1-q)$, default 0.25
$G_j^{(i)}$, $F_j^{(i)}$	`G[i, j]`, `F[i, j]`	rate $j-1 \to j$ and $j \to j-1$ from interactions alone
$g_j^{(i)}$, $f_j^{(i)}$	`g[i, j]`, `f[i, j]`	the same with noise
$\varepsilon, \varepsilon_\uparrow, \varepsilon_\downarrow$	`eps`, `eps_up`, `eps_down`	noise
$h_i(c, d)$	`donor_help(n)[c, d]`	probability that a donor of strategy i with score d helps a receiver of class c
H_{ik}	`H[i, k]`	probability that a Can of i helps a Need of k
A_{ik}	`A[i, k]`	payoff matrix, $\pi = A\rho$
π_i	`payoffs[i]`	expected payoff of strategy i
b, c	`benefit`, `cost`	benefit to the receiver, cost to the donor

Arrays are indexed from 0. The rate arrays have one unused column: `g[:, 0]` and `f[:, 0]` are 0, and `g[i, j]` (`G[i, j]`) is the rate *into* score j (so $j \geq 1$), while `f[i, j]` (`F[i, j]`) is the rate *out of* score j (also $j \geq 1$). For L scores, `g` has $L+1$ columns and `f` has L.

1.3 How the pieces fit together

chapter	pieces (module)	builds on
2 Elements	score partitions (`partitions`), strategies (`strategies`), scoring norms and noise (`norms`, `known_norms`, `noise`)	—
3 Score dynamics	population (`population`), transition rates (`rates`), the chain (`chain`), stable scores (`solver`)	elements
4 Dynamics	payoffs (`payoffs`), replicator dynamics (`replicator`), the strategy simplex and rest points (`simplex`), ESS (`ess`)	score dynamics

Everything is imported from the package: `from engine import PhiDisc, ScoringNorm, ...`. Every example in this document is run when the document is built, and what is printed below it is what the engine printed.

How each entry is laid out. *What it does*, *how it does it*, its *parameters* (name, type with the default, meaning), and an *example*.

2 Elements: classes, strategies, update rules

These pieces do not depend on each other except that strategies and norms are both written in terms of score classes.

2.1 Score partitions

module `engine/partitions.py`

2.1.1 `ScorePartition`, `PossessionPartition`, `TernaryPartition`

What it does. Splits the scores $\{0, 1, 2, \dots\}$ into classes. Strategies and norms react to a score only through its class.

How it does it. A partition is given by increasing positive *cutoffs* $x_1 < x_2 < \dots$. Class c holds the scores j with $x_c \le j < x_{c+1}$, taking $x_0 = 0$ and $x_{\text{last}+1} = \infty$; so m cutoffs give $m + 1$ classes. `classify` is a binary search (`numpy.searchsorted`) of the score among the cutoffs. Two partitions are equal when their cutoffs are. `PossessionPartition` is the cutoff list (1): class 0 is $s = 0$, class 1 is $s \ge 1$. `TernaryPartition(tau)` is $(1, \tau + 1)$: $s = 0$, $1 \le s \le \tau$, $s > \tau$.

Parameters.

name	type, default	meaning
`cutoffs`	increasing positive ints	`ScorePartition`: the first score of every class except class 0. Raises `ValueError` if not strictly increasing and positive.
`tau`	int ≥ 1	`TernaryPartition`: the last score of the middle class.

Attributes and methods: `cutoffs`, `n_classes`, `classify(j)` (class index of one score or an array of scores), `class_masses(w)` (probability mass of a score distribution in each class).

Example.

```
from engine import TernaryPartition

p = TernaryPartition(3)                 # classes: s = 0 | s = 1, 2, 3 | s >= 4
print(p.cutoffs, p.n_classes)
print(p.classify([0, 1, 3, 4, 10]))     # class index of each score
print(p.class_masses([0.2, 0.3, 0.1, 0.1, 0.1, 0.2]))
```

```
(1, 4) 3
[0 1 1 2 2]
[0.2 0.5 0.3]
```

2.2 Strategies

module `engine/strategies.py`

A strategy answers one question: *with which probability does a donor help?* It may depend on the receiver, only through the receiver's class, and on the donor's own exact score d:

$$h(c, d) = \Pr(\text{donor with score } d \text{ helps} \mid \text{receiver in class } c).$$

Every strategy supplies this table through `donor_help(n_scores)`, an array of shape (`n_classes`, `n_scores`) with entry $[c, d]$, and the partition that defines its classes.

2.2.1 `Strategy` (the base class)

What it does. Fixes what the rest of the engine can rely on: a `partition`, a `donor_help(n_scores)` table and a `name`.

How it does it. Subclasses implement `_compute_donor_help` and `name`. `donor_help` caches the table and, when more scores are asked for than have been computed, recomputes it for at least twice as many, so the cost is amortised as the score range grows.

Parameters.

name	type, default	meaning
`partition`	`ScorePartition`	the classes the strategy distinguishes in the receiver.

2.2.2 `ClassStrategy`, `BinaryStrategy`, `TernaryStrategy`

What it does. Strategies whose help probability depends only on the classes of the receiver and of the donor: a table $P[c_{\text{receiver}}, c_{\text{donor}}]$ of probabilities. `BinaryStrategy` uses the classes $s = 0$ / $s \geq 1$ (a 2×2 table), `TernaryStrategy` the classes of `TernaryPartition(tau)` (a 3×3 table).

How it does it. $h(c, d) = P[c,\ \text{class}(d)]$, evaluated for all scores at once by indexing the table with the class of every score. The *name* writes the table row by row, rows being the receiver's class and columns the donor's class, with `0` for 0, `g` for 1 ("gives") and the number otherwise: `"0,0|0,g"` helps only when both donor and receiver have a score (the Discriminators). A ternary name is followed by `(tau=T)`. `from_name` reads such a name back.

Parameters.

name	type, default	meaning
`P`	$n \times n$ array, entries in $[0, 1]$	the table; $n = 2$ binary, $n = 3$ ternary. `ValueError` for a wrong shape or an entry outside $[0, 1]$.
`tau`	int ≥ 1	`TernaryStrategy` only: the cutoff between the middle and top classes.
`name`	str	`from_name(name)` / `from_name(name, tau)`: a table written as above.
`strategy`	`BinaryStrategy`	`TernaryStrategy.from_binary(strategy, tau)`: the ternary strategy that treats both positive classes like $s \geq 1$.

Example.

```
from engine import BinaryStrategy

disc = BinaryStrategy.from_name("0,0|0,g")   # helps only if both donor and receiver have a score
print(disc.name)
print(disc.P)                                # P[receiver class, donor class]
```

```
print(disc.donor_help(4))                    # [receiver class, donor score]
mixed = BinaryStrategy([[0, 0.5], [0.25, 1]])
print(mixed.name)
```

```
0,0|0,g
[[0. 0.]
 [0. 1.]]
[[0. 0. 0. 0.]
 [0. 1. 1. 1.]]
0,0.5|0.25,g
```

```
from engine import TernaryStrategy

s = TernaryStrategy.from_name("0,0,0|0,g,0.5|0,g,g", tau=2)
print(s.name)
print(s.partition.classify([0, 1, 2, 3, 4]))
print(s.donor_help(5))                       # rows: receiver in class 0, 1, 2
```

```
0,0,0|0,g,0.5|0,g,g (tau=2)
[0 1 1 2 2]
[[0.  0.  0.  0.  0. ]
 [0.  1.  1.  0.5 0.5]
 [0.  1.  1.  1.  1. ]]
```

2.2.3 PhiDisc

What it does. The strategy "help in proportion to how little you have": a donor with score $d \geq 1$ helps a receiver *with a score* with probability $d^{-\varphi}$. A donor with score 0 helps such a receiver with probability p_0 (by default 0: it never helps). A receiver with score 0 is never helped.

How it does it. Two classes ($s = 0$, $s \geq 1$). In the table $h(c, d)$: row 0 (receiver without a score) is all zeros; row 1 is $h(1, 0) = p_0$ and $h(1, d) = d^{-\varphi}$ for $d \geq 1$. With $\varphi = 0$ every donor with a score helps (cooperators towards receivers with a score); larger φ makes richer donors give less. The name is `phi-Disc(phi=1)`, or `phi-Disc(phi=1,p0=0.3)` when $p_0 \neq 0$.

Parameters.

name	type, default	meaning
`phi`	float ≥ 0	the exponent φ. `ValueError` if negative.
`p0`	float in $[0, 1]$, default `0.0`	the probability that a donor with score 0 helps a receiver with a score. `ValueError` outside $[0, 1]$.

Example.

```
from engine import PhiDisc

never = PhiDisc(phi=1)
print(never.name)
print(never.donor_help(5)[1])        # receiver with a score: j^-1 for a donor with score j, 0 at
     j = 0
sometimes = PhiDisc(phi=1, p0=0.3)   # a donor with score 0 helps with probability 0.3
print(sometimes.name)
print(sometimes.donor_help(5)[1])
print(sometimes.donor_help(5)[0])    # a receiver with score 0 is never helped
```

```
phi-Disc(phi=1)
[0.         1.         0.5        0.33333333 0.25      ]
phi-Disc(phi=1,p0=0.3)
[0.3        1.         0.5        0.33333333 0.25      ]
[0. 0. 0. 0. 0.]
```

2.2.4 Helpers: `Defectors`, `Discriminators`, `all_binary_strategies`, `all_ternary_strategies`, `strategy_from_name`

What it does. `Defectors()` never helps (`"0,0|0,0"`); `Discriminators()` helps only when donor and receiver both have a score (`"0,0|0,g"`). `all_binary_strategies()` lists the 16 deterministic binary strategies and `all_ternary_strategies(tau)` the 512 deterministic ternary ones (every entry 0 or 1), in the order 0 before `g`. `strategy_from_name` builds any strategy from its name.

How it does it. The lists run over every 0/1 assignment of the table (`itertools.product`). `strategy_from_name` decides by the shape of the text: two rows separated by `|` is binary, three rows is ternary (it needs `tau`, or a trailing `(tau=T)`), and `phi:PHI` or `phi:PHI,p0=P` (also `phi-Disc(phi=PHI,p0=P)`) is a φ-Disc.

Parameters.

name	type, default	meaning
`tau`	int	`all_ternary_strategies(tau)`, and `strategy_from_name(name, tau)` for a ternary name without `(tau=T)`.
`name`	str	the strategy's name.

Example.

```
from engine import (Defectors, Discriminators, all_binary_strategies, all_ternary_strategies,
                    strategy_from_name)

print(Defectors().name, Discriminators().name)
print(len(all_binary_strategies()), len(all_ternary_strategies(tau=2)))
print(strategy_from_name("0,g|0.5,g").name)
print(strategy_from_name("0,0,0|0,g,0.5|0,g,g (tau=2)").name)
print(strategy_from_name("phi:0.5,p0=0.2").name)
```

```
0,0|0,0 0,0|0,g
16 512
0,g|0.5,g
0,0,0|0,g,0.5|0,g,g (tau=2)
phi-Disc(phi=0.5,p0=0.2)
```

2.3 Scoring norms (the update rules)

modules `engine/norms.py`, `engine/known_norms.py`

2.3.1 `ScoringNorm`

What it does. The rule that changes scores after a meeting: by -1, 0 or $+1$ for the donor (`CN`, "Can meets Need") and for the receiver (`NC`, "Need meets Can"), depending on the classes of both and on whether help was given.

How it does it. Two arrays indexed `[receiver class, act, donor class]` with `act` $= 0$ for "declines" and 1 for "gives", each of shape $(m, 2, m)$ for m classes. The *barcode* writes each array flattened in that index order (the donor's class changes fastest) with `-`, `0`, `+`, so `"CN:00000-00 NC:00000000"` (binary, 8 characters each) reads: only `CN[1,0,1]` is nonzero, -1: a donor with a score who *declines* a receiver with a score loses one. A score of 0 cannot go down, so the entries with a donor (for `CN`) or receiver (for `NC`) in class 0 may not be `-` (`ValueError`). Ternary norms have 18 characters each and need τ.

Parameters.

name	type, default	meaning
`CN, NC`	arrays of ints in $\{-1, 0, 1\}$, shape $(m, 2, m)$	the two rules.
`partition`	`ScorePartition`, default `PossessionPartition()`	the classes the rules use.
`barcode`	str	`ScoringNorm.from_barcode(barcode, tau=None)`: `"CN:... NC:..."` (the older `CmeetsN:`/`NmeetsC:` spelling is accepted); 8 characters is binary and takes no `tau`, 18 is ternary and needs it.
`tau`	int	`to_ternary(tau)`: lifts a binary norm, both positive classes acting like $s \geq 1$.

Also: `.barcode`, `.is_ternary`, `.partition`, equality.

Example.

```
from engine import ScoringNorm

norm = ScoringNorm.from_barcode("CN:00000-00 NC:00000000")
print(norm.barcode, norm.is_ternary)
# CN[receiver class, act, donor class]; act 0 = declines, 1 = gives
print(norm.CN[1, 0, 1])      # a donor with a score who declines a receiver with a score: -1
print(norm.CN[1, 1, 1])      # ... who gives: 0
tern = norm.to_ternary(tau=2)
print(tern.barcode, tern.partition)
```

```
CN:00000-00 NC:00000000 False
-1
0
CN:0000000--0000--000 NC:000000000000000000 TernaryPartition(tau=2)
```

2.3.2 `all_norms`

What it does. Iterates over every valid norm, e.g. to search for norms with a given ESS.

How it does it. Each entry takes the values $\{0, +\}$ where a score cannot fall and $\{-, 0, +\}$ elsewhere; `itertools.product` runs over all combinations of the two rules, and an unused rule is held at all zeros. A binary rule has $2^4 3^4 = 1296$ options, a ternary rule $2^6 3^{12} = 34{,}012{,}224$, so with both rules there are 1296^2 binary norms: iterate lazily.

Parameters.

name	type, default	meaning
`tau`	int or `None`	`None`: binary norms; an int: ternary norms with that τ.
`use_CN`	bool, `True`	vary the donor rule (otherwise all zeros).
`use_NC`	bool, `True`	vary the receiver rule (otherwise all zeros).

Example.

```
from engine import all_norms

norms = list(all_norms(use_NC=False))   # every binary norm that only changes the donor
```

```
print(len(norms), norms[0].barcode)
print(norms[-1].barcode)
```

```
1296 CN:0-0-0-0- NC:00000000
CN:++++++++ NC:00000000
```

2.3.3 NamedNorm, LEADING_EIGHT, MONEY

What it does. Norms from the literature, with the strategy that is evolutionarily stable under each: the Leading Eight (Ohtsuki and Iwasa), which only change the donor, and the money norm.

How it does it. `NamedNorm` is a small immutable record (`label`, `name`, `CN`, `NC`, `strategy`) with the properties `barcode`, `title` and the method `norm(tau=None)` that builds the `ScoringNorm` (lifted to ternary classes when `tau` is given). `LEADING_EIGHT` holds L1–L8 (the strategy is `"g,0|g,g"` for L1 and L2 and `"0,0|g,g"` for the others); `MONEY` is `CN:000000++ NC:000000-`: a donor who gives to a receiver with a score gains one and the receiver loses one.

Parameters.

name	type, default	meaning
`tau`	int or `None`	`NamedNorm.norm(tau)`: lift the binary norm to a ternary one.

Example.

```
from engine import LEADING_EIGHT, MONEY, ScoringNorm

for n in LEADING_EIGHT[:3]:
    print(n.title, "|", n.barcode, "|", n.strategy)
stern = LEADING_EIGHT[5]
print(stern.title, stern.norm().barcode)          # the ScoringNorm
print(stern.norm(tau=2).barcode)                  # lifted to ternary classes
print(MONEY.title, MONEY.barcode, MONEY.strategy)
```

```
L1 | CN:00+00-+0 NC:00000000 | g,0|g,g
L2 Consistent Standing | CN:00+-0-+0 NC:00000000 | g,0|g,g
L3 Simple Standing | CN:+0+00-+0 NC:00000000 | 0,0|g,g
L6 Stern Judging CN:+00-0-+0 NC:00000000
CN:+000--0--+000--+00 NC:000000000000000000
$ Money CN:000000++ NC:000000-- 0,0|g,g
```

2.4 Noise

module `engine/noise.py`

2.4.1 Noise

What it does. Random score changes on top of the norm: with probability ε an update is replaced by a random one, upwards with probability $\varepsilon_\uparrow$ and downwards with $\varepsilon_\downarrow$.

How it does it. An immutable record that checks its values. The transition rates G, F from interactions alone become

$$g = (1-\varepsilon)\,G + \varepsilon\,\varepsilon_\uparrow, \qquad f = (1-\varepsilon)\,F + \varepsilon\,\varepsilon_\downarrow$$

(applied in `TransitionRates`, section 3.2; a score of 0 never goes down). `ValueError` if a value is outside $[0,1]$ or $\varepsilon_\uparrow + \varepsilon_\downarrow > 1$ (the notes take the sum to be 1; a sum below 1 just leaves some noise doing nothing). `Noise.from_noise_val(v)` is $\varepsilon = v$, $\varepsilon_\uparrow = \varepsilon_\downarrow = \frac{1}{2}$.

Parameters.

name	type, default	meaning
`eps`	float in $[0,1]$, `0.0`	the noise level ε.
`eps_up`	float in $[0,1]$, `0.5`	share of noise that moves a score up.
`eps_down`	float in $[0,1]$, `0.5`	share of noise that moves a score down.

Example.

```
from engine import Noise

noise = Noise(eps=0.1, eps_up=0.4, eps_down=0.6)
print(noise)
try:
    Noise(eps=0.1, eps_up=0.7, eps_down=0.6)
except ValueError as error:
    print("ValueError:", error)
```

```
Noise(eps=0.1, eps_up=0.4, eps_down=0.6)
ValueError: eps_up + eps_down must not exceed 1 (the notes take it to be 1); got 0.7 + 0.6.
```

3 Score dynamics: from strategies and a norm to stable score distributions

3.1 Population

module `engine/population.py`

3.1.1 `Population` and `score_array`

What it does. `Population` is a list of strategies with their frequencies ρ. `score_array` turns the *initial* score distributions into the array w that the score dynamics work on.

How it does it. `Population` checks that ρ has one non-negative entry per strategy and sums to 1 (equal frequencies by default). `score_array` accepts either one distribution, shared by all n strategies, or one per strategy, pads them with zeros to a common length L and checks that each is non-negative and sums to 1; it returns the $n \times L$ array `w[i, j]`.

Parameters.

name	type, default	meaning
`strategies`	list of `Strategy`	the strategies; at least one.
`rho`	array of n floats, default equal	the frequencies; must sum to 1.
`w_init`	1-D or 2-D array-like	`score_array(w_init, n)`: one distribution, or one row per strategy.

Attributes: `strategies`, `rho`, `n`.

Example.

```
from engine import Defectors, PhiDisc, Population
from engine.population import score_array

pop = Population([Defectors(), PhiDisc(1)], rho=[0.7, 0.3])
print(pop, pop.n)
```

```
print(score_array([0.5, 0.5], 2))                # one distribution shared by both strategies
print(score_array([[1.0], [0.5, 0.5]], 2))       # one each, padded with zeros
```

```
Population(0,0|0,0: 0.7, phi-Disc(phi=1): 0.3) 2
[[0.5 0.5]
 [0.5 0.5]]
[[1.  0. ]
 [0.5 0.5]]
```

3.2 Transition rates

module `engine/rates.py`

3.2.1 `role_factor`

What it does. The probability $\kappa = q(1-q)$ that a given ordered pair of roles (this individual Can, the other Need) occurs, when each individual can help with probability q. `ROLE_FACTOR` $= 0.25$ is the value for $q = \frac{1}{2}$.

Parameters.

name	type, default	meaning
`q`	float in $(0, 1)$	the probability that an individual can help. `ValueError` otherwise.

3.2.2 `TransitionRates` and `Rates`

What it does. Given a population and its current score distributions, computes the rates at which a score of each strategy moves up and down: G, F from interactions alone, and g, f with noise. Returns them in a `Rates` record.

How it does it. Take an individual of strategy i with score s. Let $x(\cdot)$ be a score's class under the norm, and let

$$W_c = \text{mass of the population in class } c, \qquad B_{c,x} = \sum_k \rho_k \sum_{d \in x} w_d^{(k)}\, h_k(c, d)$$

(the mass of Cans in norm class x who help a Need of class c), and $M_x = \sum_k \rho_k \sum_{d \in x} w_d^{(k)}$.

- *As the Can*, it meets a Need of class c with probability W_c and helps with probability $h_i(c, s)$. Its score goes up with probability $U_s^{\text{can}} = \sum_c W_c \big[(1 - h_i(c, s))\,[CN_{c,0,x(s)} = +1] + h_i(c, s)\,[CN_{c,1,x(s)} = +1]\big]$ and down with D_s^{can}, the same with -1.
- *As the Need*, in class $a = x(s)$, it meets a Can of class x who helps with total weight $B_{a,x}$ and declines with weight $M_x - B_{a,x}$: $U_s^{\text{need}} = \sum_x \big[(M_x - B_{a,x})\,[NC_{a,0,x} = +1] + B_{a,x}\,[NC_{a,1,x} = +1]\big]$, and D_s^{need} with -1.

The rates are the role factor times the sums:

$$G_{s+1}^{(i)} = \kappa\,(U_s^{\text{can}} + U_s^{\text{need}}), \qquad F_s^{(i)} = \kappa\,(D_s^{\text{can}} + D_s^{\text{need}}),$$

and with noise $g = (1-\varepsilon)G + \varepsilon\varepsilon_\uparrow$, $f = (1-\varepsilon)F + \varepsilon\varepsilon_\downarrow$ (with $g_0 = f_0 = 0$). The code does not loop over scores and classes: it works on the coarsest partition of the scores that refines the norm's classes and every strategy's receiver classes, so each call is a few array operations whatever the strategies are, and the class tables are cached per set of strategies.

Parameters.

name	type, default	meaning
`norm`	`ScoringNorm`	the update rule.
`noise`	`Noise`	the noise.
`role_factor`	float, `0.25`	κ (see `role_factor(q)`).
`population`	`Population`	the call `rates(population, w)`.
`w`	array (n, L)	the current score distributions; the top score should be empty (see `chain.extend`).

The returned `Rates` holds `G` and `g` of shape $(n, L+1)$ and `F` and `f` of shape (n, L).

Example.

```
import numpy as np
from engine import Noise, PhiDisc, Population, ScoringNorm, TransitionRates

norm = ScoringNorm.from_barcode("CN:00000-0+ NC:00000000")
rates = TransitionRates(norm, Noise(eps=0.1))
pop = Population([PhiDisc(1)])
w = np.array([[0.5, 0.3, 0.2, 0.0]])             # score distribution, last score empty
r = rates(pop, w)
np.set_printoptions(precision=4, suppress=True)
print("G", r.G[0])     # G[j]: rate j-1 -> j from interactions
print("F", r.F[0])     # F[j]: rate j -> j-1
print("g", r.g[0])     # with noise
print("f", r.f[0])
```

```
G [0.     0.     0.125  0.0625 0.0417]
F [0.     0.     0.0625 0.0833]
g [0.     0.05   0.1625 0.1063 0.0875]
f [0.     0.05   0.1063 0.125 ]
```

3.3 The birth–death chain

module `engine/chain.py`

3.3.1 `extend`, `change`, `stationary`

What it does. `change` gives the change of the score distributions in one step of the chain, `extend` makes room for mass to move up, and `stationary` gives the stationary distribution of a chain whose rates do not change.

How it does it. With the rates of section 3.2, the change of w_j in one step is

$$\Delta w_j = -w_j\,(g_{j+1} + f_j) + w_{j-1}\,g_j + w_{j+1}\,f_{j+1}$$

(outflow up and down, inflow from below and from above), which sums to zero. It needs the top score to be empty so that the up-flow has somewhere to go: `extend` appends an empty score when the top one has mass. For constant rates detailed balance gives the stationary distribution $\pi_j \propto \prod_{s=1}^{j} g_s/f_s$, computed with logarithms for stability and normalised.

Parameters.

name	type, default	meaning
`w`	array (n, L)	score distributions (`change` needs `w[:, -1]` to be 0).
`g`	array $(n, L+1)$	up-rates, `g[:, j]` from $j-1$ to j.
`f`	array (n, L)	down-rates, `f[:, j]` from j to $j-1$.
`g, f`	1-D arrays of equal length $S+1$	`stationary(g, f)`: all entries from index 1 positive.

Example.

```
import numpy as np
from engine import chain

np.set_printoptions(precision=4, suppress=True)
w = np.array([[0.5, 0.3, 0.2]])
w = chain.extend(w)                              # mass at the top score, so an empty score is
    added above it
print(w)
g = np.array([[0.0, 0.4, 0.3, 0.2, 0.1]])        # g[j]: rate j-1 -> j (g[0] unused), one more
    than w has scores
f = np.array([[0.0, 0.1, 0.2, 0.3]])             # f[j]: rate j -> j-1 (f[0] unused)
dw = chain.change(w, g, f)
print(dw, dw.sum())                              # the change sums to 0: probability is conserved
print(chain.stationary([0.0, 0.4, 0.3, 0.2], [0.0, 0.1, 0.2, 0.3]))   # stationary distribution
    of a fixed chain
```

```
[[0.5 0.3 0.2 0. ]]
[[-0.17  0.12  0.01  0.04]] 0.0
[0.0667 0.2667 0.4    0.2667]
```

3.4 Stable score distributions

module `engine/solver.py`

3.4.1 `StableScoreSolver` and `StableScores`

What it does. Finds the stable score distribution of every strategy in a population, by running all the score chains together until they stop changing. Returns a `StableScores` record.

How it does it. Because each chain's rates depend on every strategy's distribution, the chains are stepped together:

1. Start from `w = score_array(w_init)`.
2. Repeat up to `max_iter` times: `extend` w; compute the rates from it; add the change ($w \leftarrow w + \Delta w$, one unit of time); renormalise each row to sum 1; if `tail_tol` > 0 drop the top scores whose mass is below it in every chain.
3. Stop early when $\max|\Delta w| \leq$`tol` (*converged*), or when every chain has lost all its mass in its initial scores (*transient*: the chain has left where it started and keeps spreading), unless `force_max_iter`.

The score range grows as needed; there is no maximum score. At the end the solver checks for *drift*: above the highest class cutoff the rates no longer change with the score, so if the up-rate at the top score is at least the down-rate there is no stable distribution, however small the last change was. Then `converged` is false. The record keeps `w` (the result), `w_prev` (the distributions the last rates were computed from) and `rates` (those rates).

Parameters.

name	type, default	meaning
`norm`	`ScoringNorm`	the update rule.
`noise`	`Noise`, default `Noise()`	the noise.
`tol`	float, `1e-5`	converged when the largest change in one step is at most this.
`max_iter`	int, `10000`	the most steps.
`stop_when_transient`	bool, `True`	stop when every chain has left its initial scores.
`force_max_iter`	bool, `False`	never stop early: always take `max_iter` steps.
`record_history`	bool, `False`	keep `(w, rates)` after every step in `history`.
`role_factor`	float, `0.25`	κ.
`tail_tol`	float, `0.0`	if positive, drop top scores with mass below it (keeps long runs short).
`population, w_init`		the call `solver(population, w_init)`.

`solver.with_options(**options)` returns a copy with `tol`, `max_iter`, ... changed (not the norm, noise or role factor). The result has: `w` ($n \times L$), `w0` (the fraction with score 0), `w_prev`, `rates`, `n_iter`, `converged`, `transient`, `drifting` (one flag per strategy), `history`.

Example.

```
import numpy as np
from engine import Noise, PhiDisc, Population, ScoringNorm, StableScoreSolver

np.set_printoptions(precision=4, suppress=True)
norm = ScoringNorm.from_barcode("CN:00000-0+ NC:00000000")
solver = StableScoreSolver(norm, Noise(eps=0.1, eps_up=0.3, eps_down=0.7), tol=1e-10, max_iter
    =5000)
result = solver(Population([PhiDisc(1)]), [0.5, 0.5])
print(result.converged, result.drifting, result.n_iter)
print(result.w[0][:6])
print(result.w.sum(), result.w0)
```

```
True [False] 622
[0.4066 0.1743 0.2083 0.1268 0.0555 0.0199]
1.0 [0.4066]
```

4 Dynamics: payoffs, replicator dynamics, rest points and ESS

4.1 Payoffs

module `engine/payoffs.py`

4.1.1 `Payoffs`

What it does. Turns score distributions into expected payoffs: the probability that a Can of one strategy helps a Need of another, and from it each strategy's payoff in the donation game.

How it does it. Let $P_k(c)$ be the fraction of strategy k whose score is in class c. The probability that a Can of strategy i helps a Need of strategy k is

$$H_{ik} = \sum_c \sum_d P_k(c)\, h_i(c,d)\, w_d^{(i)},$$

computed with one `einsum` over classes and scores. A Can of i helping a Need of k costs c to i and gives b to k, so with the role factor κ

$$\pi_i = \kappa\Big(b\sum_k \rho_k H_{ki} - c\sum_k \rho_k H_{ik}\Big)$$

("helped by Cans of every strategy" minus "cost of helping Needs of every strategy"); this is the payoff per interaction, role factor included.

Parameters.

name	type, default	meaning
`benefit`	float, `2.0`	b, what a receiver gains.
`cost`	float, `1.0`	c, what a donor pays.
`role_factor`	float, `0.25`	κ.
`population, w`		`help_probabilities(population, w)` returns H; `payoffs(population, w)` returns π (w: the $n \times L$ score distributions).

Example.

```
import numpy as np
from engine import (Defectors, Noise, Payoffs, PhiDisc, Population, ScoringNorm,
    StableScoreSolver)

np.set_printoptions(precision=4, suppress=True)
norm = ScoringNorm.from_barcode("CN:00000-0+ NC:00000000")
solver = StableScoreSolver(norm, Noise(eps=0.1, eps_up=0.3, eps_down=0.7), tol=1e-10, max_iter
    =5000)
pop = Population([PhiDisc(1), Defectors()], [0.5, 0.5])
w = solver(pop, [0.5, 0.5]).w
payoffs = Payoffs(benefit=2, cost=1)
print(payoffs.help_probabilities(pop, w))        # H[i, k]: a Can of i helps a Need of k
print(payoffs(pop, w))                           # payoff of each strategy
```

```
[[0.1894 0.0651]
 [0.     0.    ]]
[0.0155 0.0163]
```

4.2 Replicator dynamics

module `engine/replicator.py`

4.2.1 `ReplicatorDynamics` and `Trajectory`

What it does. Evolves the strategy frequencies by the replicator equation

$$\dot\rho_i = \rho_i\,(\pi_i - \bar\pi), \qquad \pi = A\rho, \quad \bar\pi = \rho\cdot\pi, \qquad A_{ik} = \kappa\,(b\,H_{ki} - c\,H_{ik}),$$

with the score distributions evolving underneath, and returns the path as a `Trajectory`.

How it does it. `payoff_matrix(population, w)` gives A and `velocity(population, w)` gives $(\dot\rho, \pi)$. `run` then repeats, for `n_steps` +1 states: solve the score chains for the current ρ (section 3.4), record ρ, the payoffs, the fraction at score 0 and the mean score, compute $\dot\rho$, and take an Euler step $\rho \leftarrow \rho + \mathrm{d}t\,\dot\rho$, clipped at 0 (a strategy that overshoots is extinct) and renormalised. It stops early at a *rest point*, when $\max|\dot\rho| \le$ `rest_tol`. Time is `step·dt`. At every step the chains are restarted from `w_init`, so the payoffs are a function of ρ alone; when the chains do

not converge (`scores_converged` is false) the payoffs are those after the solver's `max_iter` steps and so depend on `w_init` and `max_iter`. With `warm_start=True` each step continues from the previous step's distributions instead.

Parameters.

name	type, default	meaning
`solver`	`StableScoreSolver`	solves the score chains at each step; its `tol` and `max_iter` apply.
`payoffs`	`Payoffs`	benefit, cost and role factor.
`dt`	float, `0.1`	the Euler step of ρ.
`warm_start`	bool, `False`	continue the score chains from the previous step.
`rest_tol`	float, `1e-9`	stop when $\max \lvert\dot{\rho}\rvert$ falls to this.
`strategies, rho, w_init`		`run(strategies, rho, w_init, n_steps, record_scores=False, progress=None)`: the strategies, the starting frequencies, the initial score distribution(s), the number of steps; `record_scores` keeps every score distribution in `Trajectory.w`; `progress(done, total)` is called after each step.

`Trajectory`: `t`, `rho`, `payoffs`, `w0`, `mean_score` (one row per recorded state), `scores_converged`, `at_rest`, `w`, `mean_payoff`.

Example.

```
import numpy as np
from engine import (Defectors, Discriminators, Noise, Payoffs, ReplicatorDynamics, ScoringNorm,
    StableScoreSolver)

np.set_printoptions(precision=4, suppress=True)
norm = ScoringNorm.from_barcode("CN:00000-00 NC:00000000")      # lose a score for declining a
    receiver with a score
solver = StableScoreSolver(norm, Noise(eps=0.01), tol=1e-8, max_iter=2000)
dynamics = ReplicatorDynamics(solver, Payoffs(), dt=20.0)
traj = dynamics.run([Discriminators(), Defectors()], rho=[0.3, 0.7], w_init=[0.5, 0.5], n_steps
    =5)
print(traj.t)                                    # stopped at t = 60: Defectors have gone extinct
print(traj.rho)
print(traj.payoffs)
print(traj.at_rest, traj.scores_converged)
```

```
[ 0. 20. 40. 60.]
[[0.3    0.7   ]
 [0.4427 0.5573]
 [0.7444 0.2556]
 [1.     0.    ]]
[[0.042  0.008 ]
 [0.0701 0.0089]
 [0.1257 0.0097]
 [0.171  0.0099]]
True [False False False False]
```

4.3 The strategy simplex and its rest points

module `engine/simplex.py`

4.3.1 `ReplicatorField`, `PointResult`

What it does. Evaluates the replicator's velocity $\dot{\rho}$ at *any* point ρ of the simplex, with the payoffs, helping probabilities and score distributions behind it, one point at a time or many in parallel. This is what the arrows of the phase portrait are made of.

How it does it. `evaluate(rho)` solves the score chains for that ρ (restarting from `w_init`, as `run` does), then computes A, π, $\dot{\rho} = \rho(\pi - \rho \cdot \pi)$ and H and returns them in a `PointResult`. `evaluate_many` spreads the points over worker processes (the field is sent to each worker once, so it must be picklable; with the `spawn` start method the calling script needs an `if __name__ == "__main__":` guard).

Parameters.

name	type, default	meaning
`dynamics`	`ReplicatorDynamics`	provides the solver and the payoffs.
`strategies`	list of `Strategy`	the n strategies.
`w_init`	array-like	the initial score distribution(s).
`rho`	array of n floats	`evaluate(rho)`, `velocity(rho)`.
`points, n_jobs, progress`	list, int or `None`, callable	`evaluate_many`: the points; the number of processes (`None`: one per CPU, 1: this process); `progress(done, total)`.

`PointResult`: `rho`, `velocity`, `payoffs`, `payoff_matrix`, `help_matrix`, `w`, `converged`. `velocity(rho)` returns `(velocity, converged)`.

4.3.2 `simplex_grid`

What it does. A regular grid of points on the simplex.

How it does it. All n non-negative integer counts that sum to `resolution`, divided by `resolution`. For $n = 2$ the points run evenly from e_1 to e_2.

Parameters.

name	type, default	meaning
`n`	int ≥ 1	the number of strategies.
`resolution`	int ≥ 1	points per side.

4.3.3 `find_rest_points` and `RestPoint`

What it does. Finds the rest points of a velocity field on the simplex (for $n = 2$ or 3) and says whether each is stable, unstable, a saddle or neutral.

How it does it. Vertices are always rest points. On an edge, a rest point is a sign change of $\pi_b - \pi_a = v_b/(\rho_a \rho_b)$ between grid points, refined by Brent's method. Inside the triangle ($n = 3$) a rest point starts from a grid cell where $\pi_1 - \pi_3$ and $\pi_2 - \pi_3$ both change sign and is refined by Newton's method. Stability comes from the eigenvalues of the Jacobian of the replicator restricted to the simplex, taken by finite differences placed so that every point evaluated stays in the simplex: along a face, central differences; off the face, forward differences, which equal the *invasion fitness* $\pi_k - \bar{\pi}$ of the missing strategy k. The type is `stable` if every real part is below $-$tolerance,

`unstable` if every one is above it, `saddle` if both occur and `neutral` otherwise. Segments or curves of rest points are not reported, except where a curve meets an edge.

Parameters.

name	type, default	meaning
`velocity_fn`	callable	`velocity_fn(rho)` → `(velocity, converged)`, e.g. `field.velocity`.
`n`	2 or 3	the number of strategies.
`resolution`	int, 20	grid used if `grid` is not given.
`grid, grid_velocities, grid_converged`	arrays, optional	a grid already evaluated (any resolution, any order), to avoid evaluating it again.
`xtol`	float, `1e-6`	precision of the root finding.
`neutral_tol`	float, `1e-7`	eigenvalues smaller than this (or than twice their estimated error) count as zero; use about ten times the solver's `tol` for engine fields.

Each `RestPoint` has `rho`, `kind` (`stable`, `unstable`, `saddle`, `neutral`), `eigenvalues`, `location` (`vertex`, `edge`, `interior`) and `converged`.

Example.

```
import numpy as np
from engine import (Defectors, Discriminators, Noise, Payoffs, ReplicatorDynamics,
    ReplicatorField, ScoringNorm,
                    StableScoreSolver, find_rest_points, simplex_grid)

np.set_printoptions(precision=4, suppress=True)
norm = ScoringNorm.from_barcode("CN:00000-00 NC:00000000")
solver = StableScoreSolver(norm, Noise(eps=0.01), tol=1e-8, max_iter=2000)
strategies = [Discriminators(), Defectors()]
field = ReplicatorField(ReplicatorDynamics(solver, Payoffs()), strategies, [0.5, 0.5])
point = field.evaluate([0.3, 0.7])
print(point.velocity, point.payoffs)             # d rho / dt and the payoffs at rho = (0.3, 0.7)
print(simplex_grid(2, 4))
for p in find_rest_points(field.velocity, n=2, resolution=8, neutral_tol=1e-6):
    print(p.location, p.kind, p.rho)
```

```
[ 0.0071 -0.0071] [0.042 0.008]
[[1.   0.  ]
 [0.75 0.25]
 [0.5  0.5 ]
 [0.25 0.75]
 [0.   1.  ]]
vertex stable [1. 0.]
vertex stable [0. 1.]
interior unstable [0.1307 0.8693]
```

4.4 Evolutionarily stable strategies

module `engine/ess.py`

4.4.1 `ESSFinder`

What it does. Picks out, from a list of candidate strategies, those that are evolutionarily stable against a list of invaders.

How it does it. Maynard Smith and Price: A is an ESS if for every other strategy B either $E(A,A) > E(B,A)$ (a rare B does worse than the common A), or $E(A,A) = E(B,A)$ and $E(A,B) > E(B,B)$. The finder tests this with a resident A at frequency ρ (default 0.99) against an invader B at $1-\rho$, using the payoffs of the two-strategy population (section 4.1, from the stable score distributions at the final step):

1. Skip a candidate that earns almost nothing alone ($4\pi <$ `fitness_threshold`): never-helping populations are not interesting ESS.
2. For every invader B of another name: if A's payoff is higher, go on; if lower, A is not an ESS; if equal, swap the frequencies and require that A does not do worse than B. Payoffs are compared exactly, so strategies that earn the same payoff up to rounding ($\sim 10^{-16}$, e.g. they differ only where it never matters) are told apart by the rounding.
3. Finally require $\pi_A \geq 10^{-5}$ for the candidate alone with the solver restricted to `final_max_iter` steps.

Parameters.

name	type, default	meaning
`solver`	`StableScoreSolver`	used for every population.
`payoffs`	`Payoffs`	benefit, cost, role factor.
`rho`	float, `0.99`	frequency of the resident when tested against an invader.
`fitness_threshold`	float, `1e-5`	candidates whose $4\times$ payoff alone is below this are skipped.
`final_max_iter`	int, `100`	`max_iter` of the last check.
`candidates, invaders, w_init`		`finder(candidates, invaders, w_init)` returns the candidates that are ESS against every invader; `is_ess(A, invaders, w_init)` tests one.

Also `payoff_alone(strategy, w_init)` and `payoffs_against(A, B, rho_A, w_init)`.

Example.

```
from engine import (ESSFinder, Noise, Payoffs, ScoringNorm, StableScoreSolver,
    all_binary_strategies)

norm = ScoringNorm.from_barcode("CN:00000-00 NC:00000000")      # lose a score for declining a
    receiver with a score
solver = StableScoreSolver(norm, Noise(eps=0.01), tol=1e-5, max_iter=100)
strategies = all_binary_strategies()
finder = ESSFinder(solver, Payoffs(), rho=0.99)
ess = finder(strategies, strategies, w_init=[0.5, 0.5])
print([s.name for s in ess])
```

```
['0,0|0,g']
```

A Where the pieces are used, and how to rebuild this document

where	uses
`gui/`	the local web GUI: strategies, parameters, results. Builds the strategies and norm from the settings (`gui/spec.py`), runs a `ReplicatorField` over the simplex and `find_rest_points`, and a `ReplicatorDynamics` run from a chosen point.
`scripts/ess_sweep.py`	the ESS search over many norms with `ESSFinder`, one task per norm (Slurm-friendly); `scripts/plot_ess.py` plots its result.
`experiments/`	older scripts moved onto the engine.
`tests/`	the engine against independent re-implementations, an agent-based simulation and hand calculations; `docs/verification/` reports these.

Rebuild. The examples are in `docs/engine/examples.py`; `python docs/engine/build_docs.py` runs them, writes their code and output into `docs/engine/generated/` and compiles `engine.tex`. `tests/test_docs.py` runs every example, so an example that stops working fails the tests.